

# Linguaggio Java

Programmare con gli Stream

# Introduzione agli stream

# Introduzione agli Streams

- Raggruppare e rappresentare dati è una operazione molto comune in programmazione, per questo motivo le collections sono una delle librerie più utilizzate in Java.
  - Per lo più, una volta raggruppati i dati in una collezione, si vogliono compiere operazioni di ricerca modifica basata su determinati criteri.
- **Perché introdurre nuove API per raggruppare e rappresentare i dati?**
  - In sintesi, per operare sui dati con un approccio **dichiarativo** e non programmatico

```
List<Dish> lowCaloricDishes = new ArrayList<>();  
for(Dish d: menu){  
    if(d.getCalories() < 400){  
        lowCaloricDishes.add(d);  
    }  
}
```

Java  
(SE7)

```
Select *  
From Dish  
Where calories < 400
```

SQL

Java8 introduce nel linguaggio (Java SE) il concetto di programmazione dichiarativa. La programmazione dichiarativa è l'approccio alla base dei framework e in generale della programmazione moderna

# Introduzione agli Streams

- Se pensiamo ad SQL, sicuramente il suo approccio lo rende più semplice di un linguaggio di programmazione
  - Il segreto è dichiarare ciò di cui si ha bisogno senza la necessità di indicare il modo con cui ottenerlo
- Bisogna pensare alla programmazione non più come un listato di azioni (algoritmo) ma come un insieme di indicazioni
- Gli Streams sono una nuova API introdotta con Java 8 che una in maniera significativa il concetto di programmazione dichiarativa
  - Indicare solo ciò che si vuole consente alla libreria sottostante di scegliere il modo migliore per poterlo fare. Ad esempio, in un ambiente multicore l'uso dei thread è totalmente trasparente al programmatore
  - NOTA: anche se Oracle non usa toni polemici, per quanto riguarda il parallelismo in sostanza dice:  
**meglio se i threads li creo e li gestisco io!**

# Introduzione agli Streams

- Gli Streams sono una nuova libreria per la manipolazione e la gestione dei dati in una forma dichiarativa da usare con o in alternativa alle collections.
- Come introduzione vediamo la differenza tra Java 7 e Java 8 su un esempio: dato un menù si vogliono recuperare i nomi dei piatti meno calorici ordinati per numero di calorie

```
List<Dish> lowCaloricDishes = new ArrayList<>();
for(Dish d: menu){
    if(d.getCalories() < 400){
        lowCaloricDishes.add(d);
    }
}
Collections.sort(lowCaloricDishes, new Comparator<Dish>() {
    public int compare(Dish d1, Dish d2){
        return Integer.compare(d1.getCalories(), d2.getCalories());
    }
});
List<String> lowCaloricDishesName = new ArrayList<>();
for(Dish d: lowCaloricDishes){
    lowCaloricDishesName.add(d.getName());
}
```

Filtriamo i piatti in base alle calorie

ordiniamoli in base ad un criterio

Prendiamo solo i nomi dei piatti

# Introduzione agli Streams

Con Java 8 e gli Streams

```
List<String> lowCaloricDishesName =  
    menu.stream().parallel()  
        .filter(d -> d.getCalories() < 400)  
        .sorted(comparing(Dish::getCalories))  
        .map(Dish::getName)  
        .collect(toList());
```

Torniamo il  
risultato come  
una List

Prendiamo solo i  
nomi

Filtriamo i piatti in base  
alle calorie

ordiniamoli in base ad  
un criterio

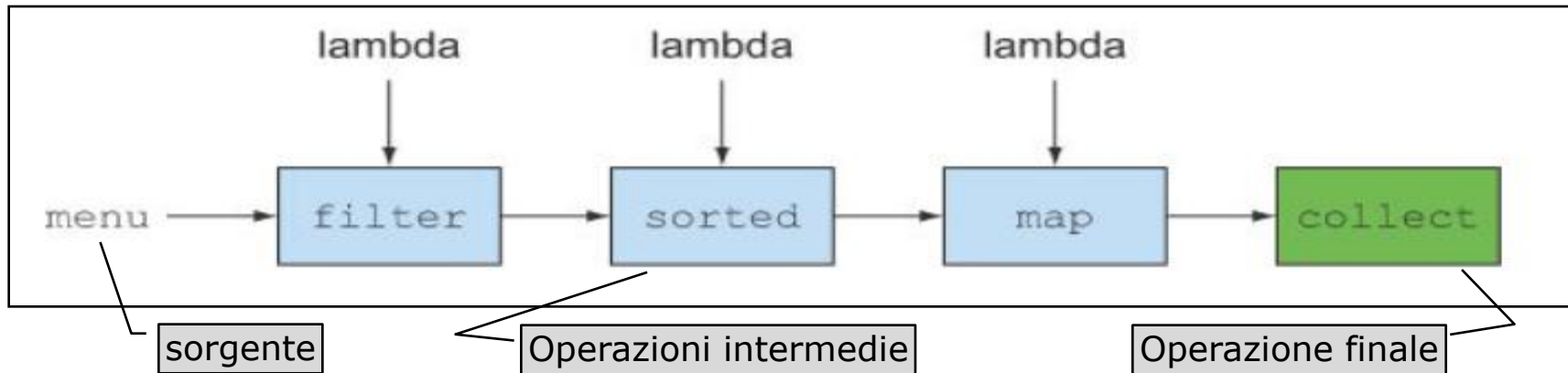
A parte l'approccio dichiarativo, possiamo notare che gli stream utilizzano il concetto di catena di operazioni (pipenile)

- Il risultato di una certa operazione è passato direttamente alla operazione successiva come in una catena di montaggio

NOTA: in questi esempi, per motivi di leggibilità, le import statiche sono state omesse (come nel caso del metodo `toList()` della classe `Collectors`)

# Introduzione agli Streams

Illustrazione della pipeline delle operazioni dal precedente esempio:



In definitiva potremmo dire che gli Stream sono:

- Dichiarativi
- Componibili
- Parallelizzabili

Java program

```
List<String> lowCaloricDishesName =  
    menu.stream()  
        .filter(d -> d.getCalories() < 400)  
        .sorted(comparing(Dish::getCalories))  
        .map(Dish::getName)  
        .collect(toList());
```

Store all the names in a List. →

← Select dishes that are below 400 calories.

← Sort them by calories.

← Extract the names of these dishes.

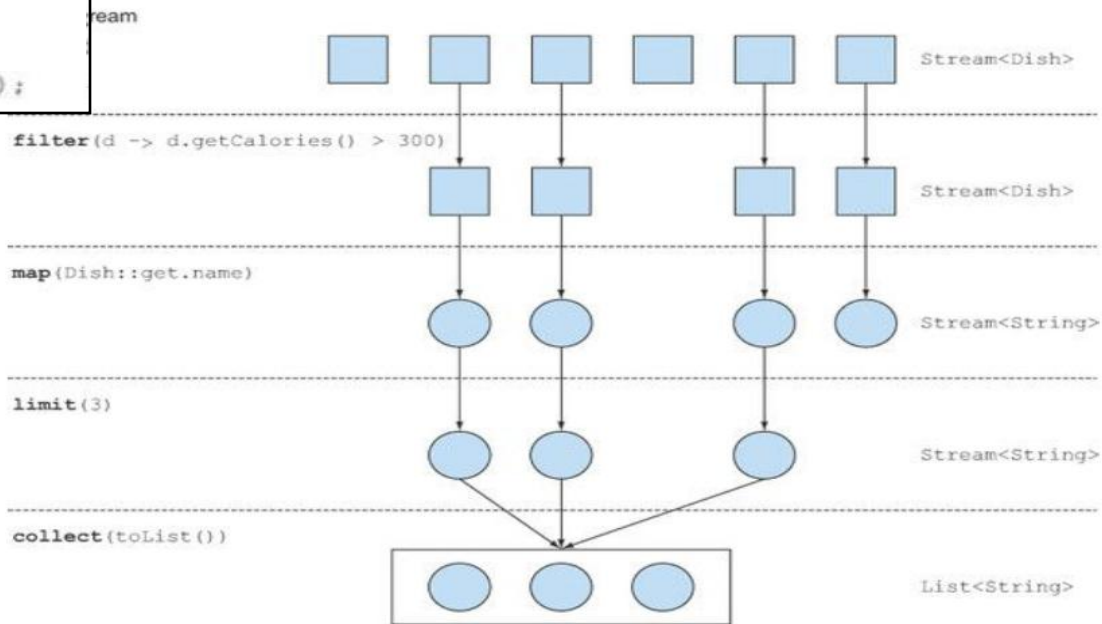
Sorgente dati



# Introduzione agli Streams

```
import static java.util.stream.Collectors.toList;
List<String> threeHighCaloricDishNames =
    menu.stream()
        .filter(d -> d.getCalories() > 300)
        .map(Dish::getName)
        .limit(3)
        .collect(toList());
System.out.println(threeHighCaloricDishNames);
```

Ecco un altro esempio



# Introduzione agli Streams

- Uno Stream è un oggetto che rappresenta una **sequenza di dati** tratti da una **sorgente** su cui si intende fare delle **operazioni**
  - **Sequenza di dati** → come per le collections, anche gli stream vengono definiti attraverso una interfaccia che ne definisce il comportamento generale
    - La differenza fondamentale tra collections e streams è che le collezioni definiscono strutture dati mentre stream definiscono operazioni sui dati
  - **Sorgente** → gli streams non sono strutture dati, bensì sono oggetti che attingono i dati da una sorgente che può essere, ad esempio, una collection, un array o una periferica
    - I dati sono sequenziali nel senso che gli stream non cambiano la sequenza originale dei dati letta dalla sorgente
    - **Qualunque operazione facciamo attraverso gli stream, la sorgente resta inalterata**
  - **Operazioni** → gli streams supportano operazioni (stile SQL) come ordinamento, raggruppamento, ricerca, filtraggio ecc... Queste operazioni possono essere eseguite sequenzialmente o in parallelo

# Esempio di sorgente inalterata

- Consideriamo il seguente esempio:

```
List<Impiegato> dipendenti = //creo e popolo una lista di Impiegati  
  
Comparator<Impiegato> c1 = (i1, i2)->i1.getMansione().compareTo(i2.getMansione());  
dipendenti.stream().sorted(c1.thenComparing(Impiegato::getNome));  
  
System.out.println(dipendenti); //stampo la sorgente originale
```

qualunque sia la manipolazione effettuata nella parte centrale tra la creazione e la stampa della sorgente (la lista dei dipendenti), non avrà effetto sulla sorgente stessa

La lista resta inalterata!

# Collection vs Stream

Una collection ed uno stream sono concettualmente cose diverse.

Una **collection** è una struttura dati che

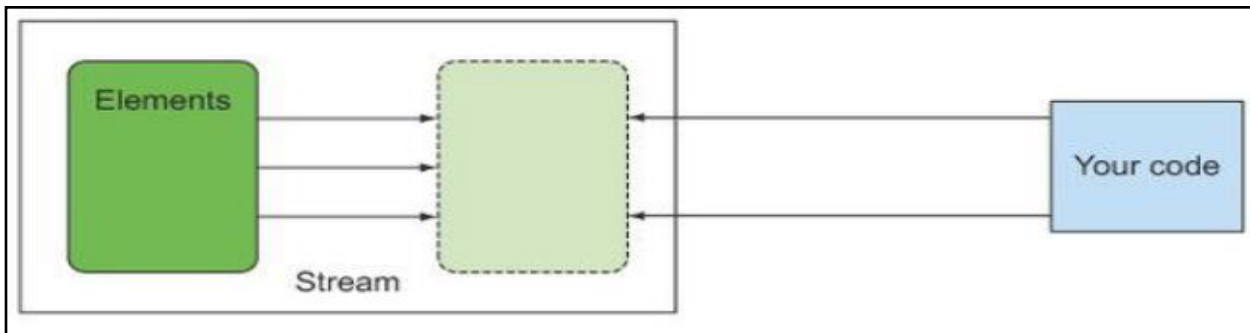
- Richiede la presenza di tutti i dati prima di iniziare una elaborazione su di essi
  - Alta occupazione di memoria
- Prevede una elaborazione nel momento in cui i dati vengono inseriti/eliminati
  - Ad esempio, un TreeSet ordina i dati al momento del loro inserimento/rimozione
- Prevede l'uso di un iteratore esterno

Uno **stream** è una struttura dati che

- Non richiede la presenza di tutti prima ma i dati vengono prelevati/generati dalla sorgente in base al loro consumo ("just-in-time" o "on demand");
  - Bassa occupazione di memoria
- I dati sono forniti come flusso sequenziale e sono consumabili
  - Se un dato è consumato non può essere recuperato nuovamente
- Prevede l'uso di un iteratore interno

# Iteratore interno vs iteratore esterno

- Differenze tra un iteratore interno ed uno esterno



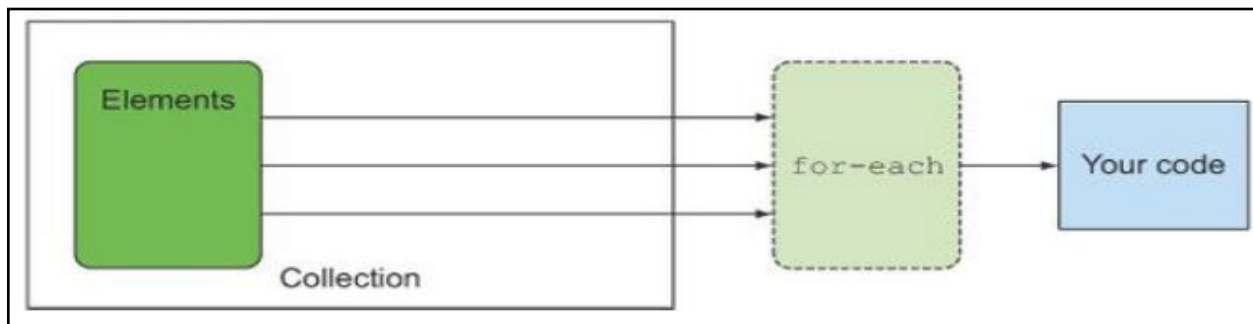
Stream: logica dichiarativa.

```
List<String> names = menu.stream()
    .filter(dish -> dish.getName().contains("apple"))
    .map(Dish::getName)
    .collect(toList());
```

Collection: logica programmatica.

```
List<String> names = new ArrayList<>();
for(Dish d: menu){
    names.add(d.getName());
}
```

← Ext  
it t



# L'architettura

Per il momento introduciamo due modalità per generare uno stream a partire dai dati. Successivamente vedremo le diverse possibili sorgenti di dati e come agganciarle

L'interfaccia `java.util.Collection` introduce due nuovi metodi per la creazione di uno stream a partire da una collezione di dati:

- **default** `Stream<E> stream()` //singolo thread
- **default** `Stream<E> parallelStream()` //multiThread

Il comportamento di default di questi metodi è ritornare uno stream come sequenza semplice di elementi della collezione

La classe `Stream` offre i metodi statici per creare la pipeline

- **static** `<T> Stream<T>empty()` → costruisce uno stream vuoto
- **static** `<T> Stream<T>of(T t)` → costruisce uno stream con un singolo valore
- **static** `<T> Stream<T>of(T... values)` → costruisce uno stream con 0 o più valori

# Stream: operazioni

Le operazioni su uno stream sono di due tipi

- **Operazioni intermedie:** filtraggio, raggruppamento, limitazione e selezione
- **Operazione finale:** stoccaggio, ricerca, riduzione

Le prime possono essere collegate a formare un pipeline coerente di operazioni sui dati mentre la seconda serve per chiudere la pipeline ed avviare l'effettiva esecuzione

**Le operazioni intermedie** sono lazy: vengono processate solo al momento della operazione finale in modo da ottimizzare il processo

- Anche se per noi le operazioni intermedie sono concettualmente separate, l'esecutore potrebbe riorganizzarle o fonderle per ottimizzare il processo
- le operazioni intermedie vengono eseguite **solo se** vi è e viene eseguita una operazione finale

**Le operazioni finali** chiudono lo stream generando un risultato.

- Una volta eseguita l'operazione finale, lo stream va considerato chiuso e non è possibile compiere ulteriori azioni
  - `java.lang.IllegalStateException`: stream has already been operated upon or close

# Stream: operazioni

- Le operazioni intermedie sono:

| Operazione | Tipo di ritorno | Argomenti operazione | Descrizione chiamata lambda    |
|------------|-----------------|----------------------|--------------------------------|
| filter     | Stream<T>       | Predicate<T>         | $T \rightarrow \text{boolean}$ |
| map        | Stream<R>       | Function<T,R>        | $T \rightarrow R$              |
| flatMap    |                 |                      |                                |
| limit      | Stream<T>       | long                 |                                |
| sorted     | Stream<T>       | Comparator<T>        | $(T,T) \rightarrow \text{int}$ |
| distinct   | Stream<T>       |                      |                                |
| peek       | Stream<T>       | Consumer<? Super T>  | $() \rightarrow T$             |
| skip       | Stream<T>       | long                 |                                |



# Operazione intermedia: filter

- Una operazione molto comune è il filtraggio dei dati in base a dei criteri; la classe Stream offre i metodi filter() e distinct() a tale scopo
  - `filter(Predicate<? super T> predicate)` → ritorna uno stream i cui elementi corrispondono ai criteri definiti dal predicato, scarta i restanti

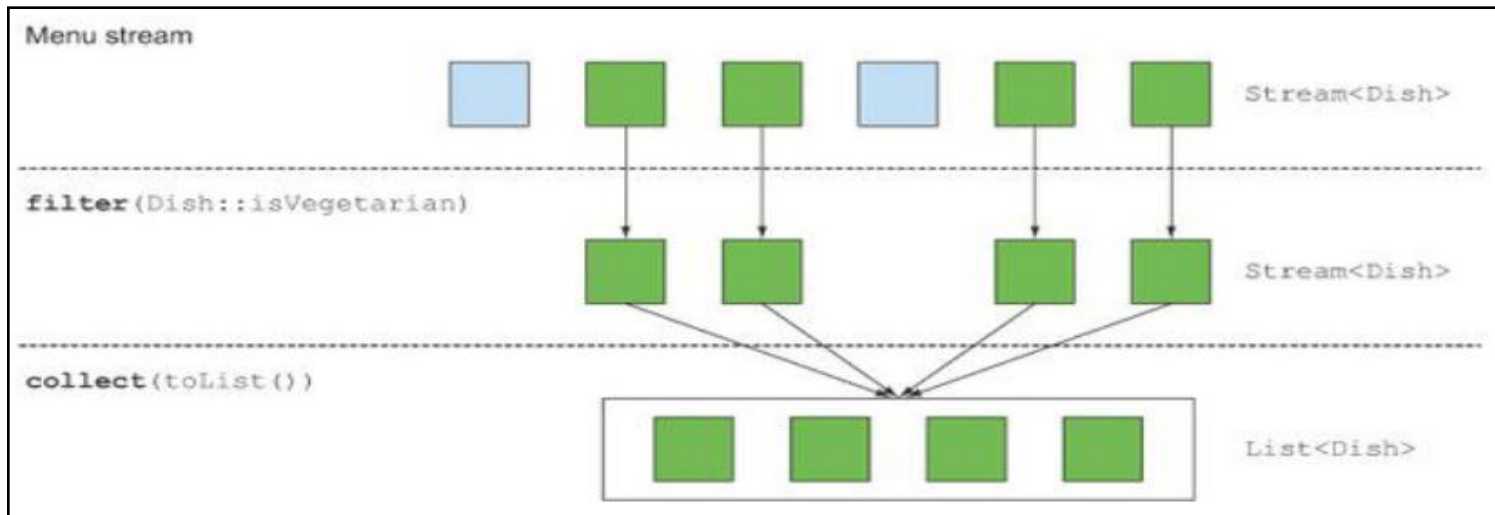
se ci fossero dei dati duplicati si possono eliminare con distinct()

- `Stream<T> distinct()` → ritorna uno stream i cui elementi sono tutti diversi eliminando eventuali oggetti uguali in accordo con il metodo equals()

# Operazione intermedia: filter

- Esempio di filtraggio gli elementi di uno stream attraverso un predicato.
  - Il risultato è un secondo stream con solo gli elementi filtrati

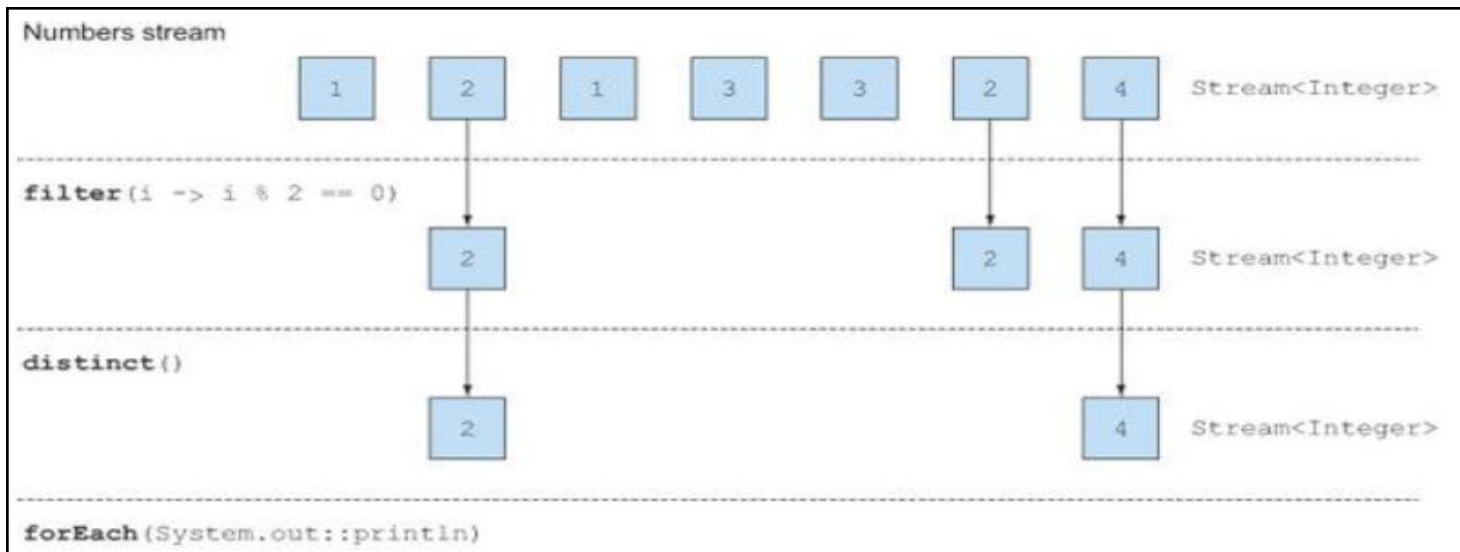
```
List<Dish> vegetarianMenu = menu.stream()  
                                .filter(Dish::isVegetarian)  
                                .collect(toList());
```



# Operazione intermedia: distinct

- Esempio di eliminazione dei doppi dopo una operazione di filtraggio attraverso l'operazione distinct

```
List<Integer> numbers = Arrays.asList(1, 2, 1, 3, 3, 2, 4);  
numbers.stream()  
    .filter(i -> i % 2 == 0)  
    .distinct()  
    .forEach(System.out::println);
```



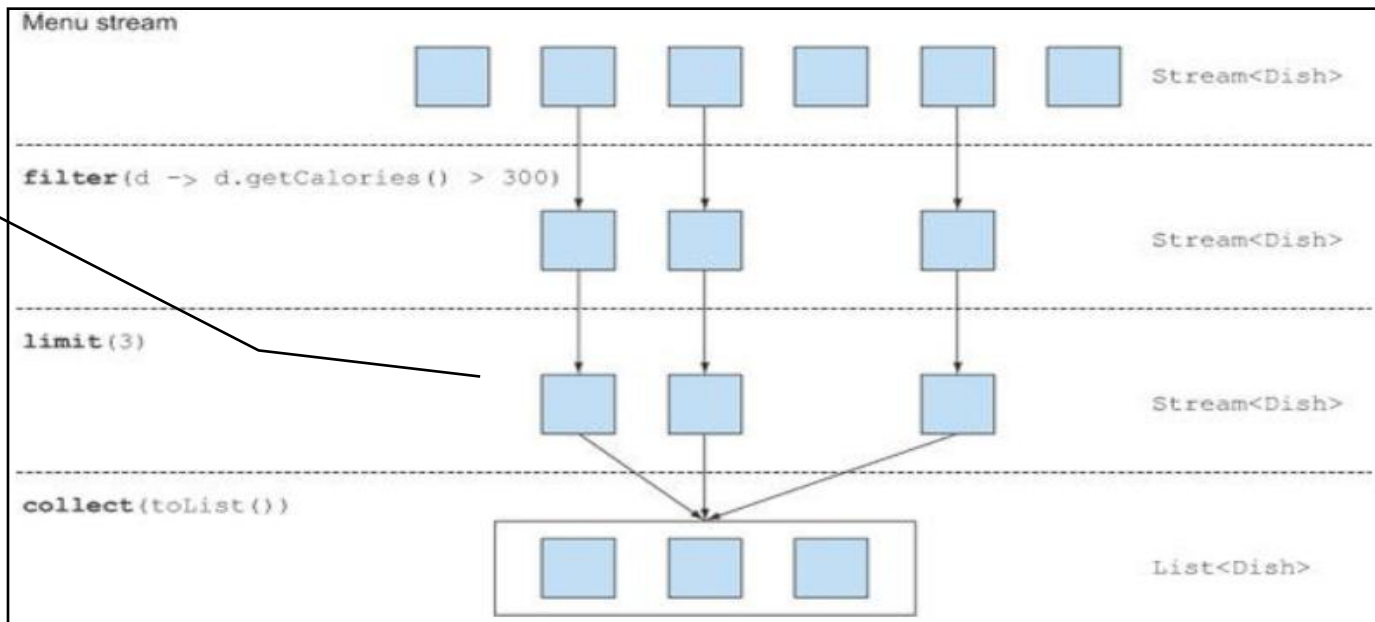
# Operazione intermedia: limit

- E' possibile limitare il numero di elementi attraverso limit

```
List<Dish> dishes = menu.stream()  
    .filter(d -> d.getCalories() > 300)  
    .limit(3)  
    .collect(toList());
```

limit è performante:  
non appena 3 elementi  
sono disponibili,  
l'operazione termina e  
si passa allo step  
successivo (collect)

Nota che gli stream  
rispettano la sequenza  
definita dalla sorgente.  
Ad esempio se la  
sorgente è un Set,  
l'ordine finale degli  
elementi non è  
prevedibile



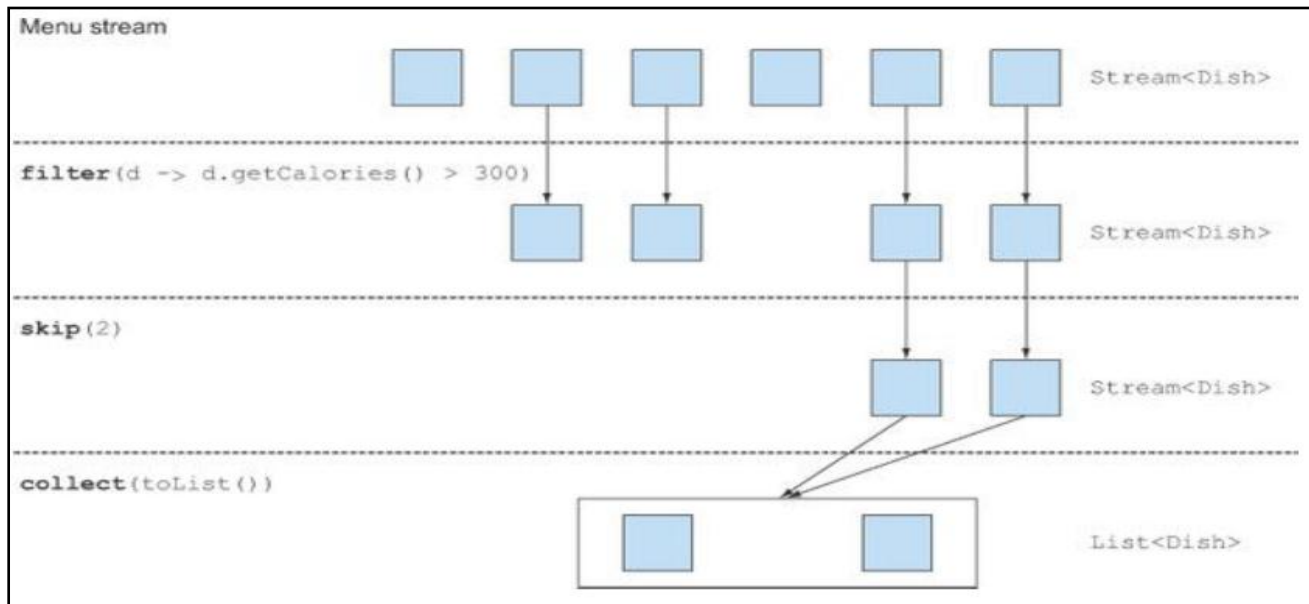
# Operazione intermedia: limit e skip

- Limit e skip sono operazioni molto pericolose in termini di performance
  - Se lo stream non parallelo queste operazioni non comportano particolari problemi
  - Con uno stream parallelo invece possono essere particolarmente costose (soprattutto con un parametro molto elevato). Questo perché limit non deve tornare giusto n elementi ma li deve tornare anche nell'ordine sequenziale dello stream. Così skip deve saltare i primi n elementi e non giusto n elementi
    - gli stream rispettano la sequenza definita dalla sorgente
- le performance aumenterebbero in maniera significativa se lo stream fosse "unordered"

# Operazione intermedia: skip

- E' possibile scartare un certo numero di elementi iniziale

```
List<Dish> dishes = menu.stream()  
    .filter(d -> d.getCalories() > 300)  
    .skip(2)  
    .collect(toList());
```



## Operazione intermedia: map

- Il metodo map può essere anche invocato a cascata secondo la normale catena di invocazione (pipeline). Ad esempio:

```
List<Integer> dishNameLengths = menu.stream()
    .map(Dish::getName)
    .map(String::length)
    .collect(toList());
```

- si ottiene una List<Integer> con le lunghezze dei nomi di ogni piatto.
  - Su ogni elemento dello stream viene invocato il metodo getName e poi sul risultato il metodo length

# Operazione intermedia: map

Una operazione molto comune sui dati è quella di selezionare solo alcuni campi di un oggetto. Gli stream consentono tale selezione tramite i metodi `map` e `flatMap`

- Corrisponde alla clausola `SELECT` di una query `sql`

Il metodo `map` trasforma gli oggetti originali dello stream in oggetti differenti.

- La trasformazione è basata sulla funzione passata come argomento al metodo

```
List<String> dishNames = menu.stream()  
    .map(Dish::getName)  
    .collect(toList());
```

in questo esempio, al metodo `map` viene passata una funzione che invoca il metodo `getName` della classe `Dish`. L'effetto è che il metodo `getName` viene invocato su ogni elemento dello stream.

Il metodo `getName` ritorna una stringa di conseguenza, ogni oggetto `Dish` dello stream viene trasformato in oggetto `String` (mapping)

- Il risultato è una `List<String>` con i soli nomi dei piatti (`Dish`)



# Operazione intermedia: flatMap

Una piccola variazioni sul tema è l'operazione flatMap.

- FlatMap trasforma uno stream di vettori in uno stream di singoli elementi

Partendo dall'esempio precedente, data una lista di parole supponiamo di volere un elenco dei diversi caratteri presenti in queste parole, cioè eliminando i duplicati

Usando soltanto distinct non si raggiunge il risultato

```
List<Integer> dishNameLengths = menu.stream()  
    .map(word -> word.split(""))  
    .distinct()  
    .collect(toList());
```

si ottiene una List<String[]>

cioè ogni elemento della lista è un vettore di tipo String. L'operazione distinct non potrebbe al risultato sperato perché valuterebbe l'uguaglianza dei vettori e non dei suoi singoli elementi

# Operazione intermedia: flatMap

In questo caso viene in aiuto l'operazione flatMap che converte uno stream di vettori in uno stream di elementi di questi vettori

```
List<Integer> dishNameLengths = menu.stream()
    .map(word -> word.split(""))
    .flatMap(Arrays::stream)
    .distinct()
    .collect(toList());
```

si ottiene una List<String>

cioè ogni i vettori derivati dall'operazioni split vengono "fusi" in un unico stream con i loro elementi (nell'ordine in cui si presentano)

- `<R> Stream<R> flatMap(Function<? super T,? extends Stream<? extends R>> mapper)`

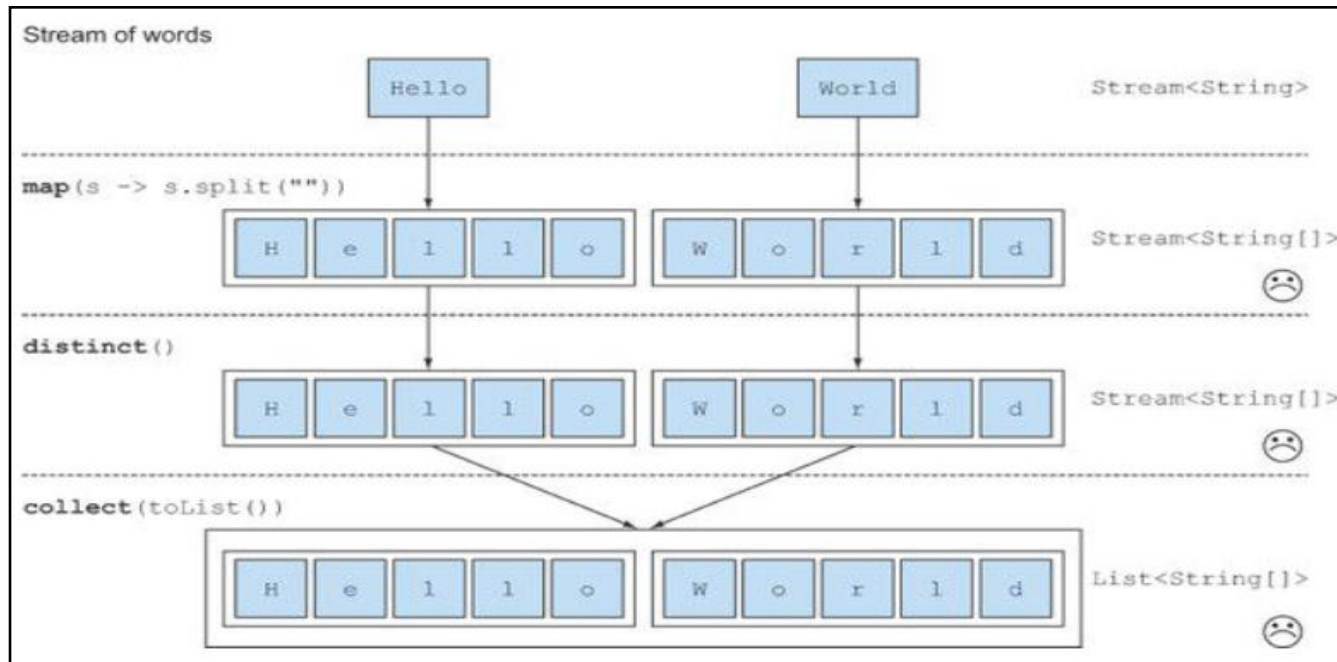
NOTA: flatMap è una operazione intermedia che ritorna uno stream

NOTA: fonde strutture come vettori in un unico stream, quindi due invocazioni consecutive portano ad un errore di compilazione

- `IntStream flatMapToInt(Function<? super T,? extends IntStream> mapper)`
- `LongStream flatMapToLong(Function<? super T,? extends LongStream> mapper)`
- `DoubleStream flatMapToDouble(Function<? super T,? extends DoubleStream> mapper)`

# Operazione intermedia: flatMap

```
List<String[]> dishNameLengths = lista.stream()  
    .map(word -> word.split(""))  
    .distinct()  
    .collect(toList());
```



# Operazione intermedia: flatMap

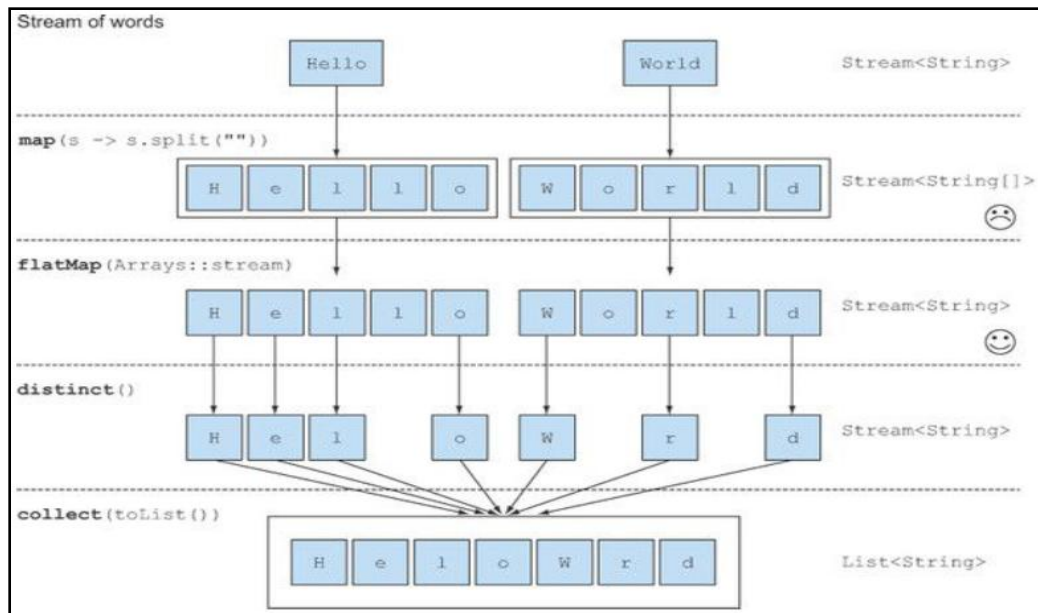
```
List<Integer> dishNameLengths = menu.stream()
    .map(word -> word.split(""))
    .flatMap(Arrays::stream)
    .distinct()
    .collect(toList());
```

Da `Stream<String>` a `Stream<String[]>`

Da `Stream<String[]>` a  
`Stream<Stream<String>>`  
**fusi in**  
`Stream<String>`

FlatMap compie 2 azioni:

1. Applicare la funzione a tutti gli elementi di uno stream  
La funzione deve trasformare gli elementi in uno stream
2. Fonde gli stream risultato in uno stream unico composto dai singoli elementi



# Operazione intermedia: peek

Nato principalmente a scopo di debug, utile se si vuole osservare lo stato degli oggetti in un certo punto della pipeline.

In generale applica un certo comportamento su ogni elemento dello stream

```
List<String> list = new ArrayList<>();  
list.add("one"); list.add("two"); list.add("three"); list.add("four");  
  
list.stream()  
    .filter(e -> e.length() > 3)  
    .peek(e -> System.out.println("Filtered value: " + e))  
    .map(String::toUpperCase)  
    .peek(e -> System.out.println("Mapped value: " + e))  
    .collect(Collectors.toList());
```

ritorna i valori [ THREE, FOUR]

Grazie al metodo peek() stampiamo il risultato delle operazioni intermedie fatte da filter e map

- `Stream<T> peek(Consumer<? super T> action)`

# Operazione intermedia: sorted

- Altra operazione molto comune è l'ordinamento. Sorted consente di ordinare gli elementi di uno stream in base a qualche criterio
- In questo esempio

```
List<String> list = new ArrayList<>();  
list.add("one"); list.add("two"); list.add("three"); list.add("four");  
  
list.stream()  
    .peek(e -> System.out.println("original value: " + e))  
    .sorted((s1, s2) -> s1.compareTo(s2)).  
    // .sorted().  
    .peek(e -> System.out.println("Mapped value: " + e))  
    .collect(Collectors.toList());
```

- si ottiene una lista ordinata in ordine alfabetico [four, one, three, two]
- I metodi sono
  - [Stream<T>sorted\(\)](#) → ordina in accordo con l'ordinamento naturale degli elementi
  - [Stream<T>sorted\(Comparator<? super T> comparator\)](#)

# Esercizio

```
class Veicolo{
    private int id;
    private String nome;
    public Veicolo(int id, String nome) {
        this.id = id;
        this.nome = nome;
    }
    public int getId() {return id;}
    public String getNome() {return nome;}

    public String toString() {return nome;}
}
```

```
List<Veicolo> l = Arrays.asList(
    new Veicolo (2,"Carro"),
    new Veicolo (3,"Bike"),
    new Veicolo (1,"Tir"));
l.stream().
// inserisci il codice qui
forEach(System.out::print);
```

Scegliere 2 possibili soluzioni per stampare TirCarroBike

1. `sorted((v1, v2) -> Integer.compare(v1.getId(), v2.getId()))`
2. `map(v -> v.getId()).sorted()`
3. `sorted(Comparator.comparing((Veicolo v) -> v.getId()))`
4. `sorted((v1, v2) -> v1.getId()<v2.getId())`
5. `sorted(Comparable.comparing(Veicolo::getNome)).reversed`

# Esercizio

```
class Veicolo{
    private int id;
    private String nome;
    public Veicolo(int id, String nome) {
        this.id = id;
        this.nome = nome;
    }
    public int getId() {return id;}
    public String getNome() {return nome;}

    public String toString() {return nome;}
}
```

```
List<Veicolo> l = Arrays.asList(
    new Veicolo (2,"Carro"),
    new Veicolo (3,"Bike"),
    new Veicolo (1,"Tir"));
l.stream().
// inserisci il codice qui
forEach(System.out::print);
```

Scegliere 2 possibili soluzioni per stampare TirCarroBike

1. `sorted((v1, v2) -> Integer.compare(v1.getId(), v2.getId()))`
2. `map(v -> v.getId()).sorted()` *//ordina ma sul campo id*
3. `sorted(Comparator.comparing((Veicolo v) -> v.getId()))`.
4. `sorted((v1, v2) -> v1.getId()<v2.getId())`. *//il tipo di ritorno deve essere int*
5. `sorted(Comparable.comparing(Veicolo::getNome)).reversed` *//comparing non esiste*



# Stream: operazioni finali

| Operazione | Tipo di ritorno | Argomenti operazione | Descrizione chiamata lambda     |
|------------|-----------------|----------------------|---------------------------------|
| anyMatch   | boolean         | Predicate<T>         | $T \rightarrow \text{boolean}$  |
| allMatch   | boolean         | Predicate<T>         | $T \rightarrow \text{boolean}$  |
| noneMatch  | boolean         | Predicate<T>         | $T \rightarrow \text{boolean}$  |
| findAny    | Optional<T>     |                      |                                 |
| findFirst  | Optional<T>     |                      |                                 |
| forEach    | void            | Consumer             | $T \rightarrow \text{void}$     |
| collect    | R               | Collector<T,A,R>     |                                 |
| reduce     | Optional<T>     | BinaryOperator<T>    | $(T, T) \rightarrow T$          |
| count      | long            |                      |                                 |
| max        | Optional<T>     | Comparator<T>        | $(T, T) \rightarrow \text{int}$ |
| min        | Optional<T>     | Comparator<T>        | $(T, T) \rightarrow \text{int}$ |
| average *  | OptionalDouble  |                      |                                 |

\* disponibile solo nelle classi specializzate come IntStream, LongStream e DoubleStream

# Operazione finale: stoccaggio dei dati

L'operazione finale più semplice è quella di raccogliere i dati rimasti dopo le operazioni intermedie della pipeline e riottenerli sotto forma di Collection.

A tale scopo, gli Stream mettono a disposizione il metodo **collect()**

- [<R,A> R collect\(Collector<? super T,A,R> collector\)](#)
- [<R> R collect\(Supplier<R> supplier, BiConsumer<R,? super T> accumulator, BiConsumer<R,R> combiner\)](#)

Ad esempio:

```
List<Integer> dishNameLengths = menu.stream()
    .map(Dish::getName)
    .map(String::length)
    .collect(Collectors.toList());
```

ritorna una collection di tipo List<Integer> con i valori lavorati dalle operazioni intermedie map e poi ancora map

# Operazione finale: ricerche

- Una delle operazioni più comuni e richieste è la ricerca. Le operazioni finali di tipo match sono utili per verificare o cercare uno o più elementi in uno stream che corrispondono ad un certo criterio.
- Per le semplici verifiche la classe Stream mette a disposizione:
  - **boolean anyMatch(Predicate<? super T> predicate)** → restituisce true se lo stream contiene almeno una occorrenza che corrisponde al criterio di ricerca impostato
  - **boolean allMatch(Predicate<? super T> predicate)** → restituisce true se tutti gli elementi dello stream corrispondono al criterio di ricerca impostato
  - **boolean noneMatch(Predicate<? super T> predicate)** → restituisce true se tutti gli elementi dello stream NON corrispondono al criterio di ricerca impostato

Per semplice verifica si intende il fatto di sapere se nello stream vi sono elementi che corrispondono al criterio impostato ma non quali sono questi elementi

# Operazione finale: ricerche

## Esempio di uso di anyMatch

```
if(menu.stream().anyMatch(Dish::isVegetarian)){  
    System.out.println("The menu is (somewhat) vegetarian friendly!!");  
}
```

restituisce true se almeno una pietanza è marcata come vegetariana

## Esempio di uso di allMatch

```
if(menu.stream().allMatch(d -> d.getCalories() < 1000){  
    System.out.println("Yeah! All Dishes are for me!");  
}
```

restituisce true se tutte le pietanze hanno una quantità di calorie minore di 1000

## Esempio di uso di noneMatch

```
if(menu.stream().noneMatch(d -> d.getCalories() >= 1000)){  
    System.out.println("Yeah! All Dishes are for me!");  
}
```

come il precedente, restituisce true se tutte le pietanze hanno una quantità di calorie minore di 1000

# Operazione finale: ricerche

Le operazioni precedenti sono utili ma ritornano un boolean. La logica quindi è quella del match. Esiste anche la possibilità di cercare ed ottenere uno o più elementi che corrispondono ai criteri impostati

Per le ricerche le operazioni sono:

- **Optional<T>findFirst()** → ritorna un elemento (il primo) che corrisponde al criterio di ricerca impostato. **Quindi ha un comportamento deterministico**
- **Optional<T>findAny()** → ritorna un elemento (uno qualsiasi) che corrisponde al criterio di ricerca impostato. **Quindi ha un comportamento non deterministico.**
  - Invocando più volte `findAny()` si potrebbero avere risultati sempre diversi

## **NOTA: tutte le operazioni di ricerca sono cortocircuitate.**

Se durante l'operazione di match si deduce che il risultato sarà comunque true o false, evita di esaminare i restanti elementi dello stream

Se, ad esempio, durante l'operazione di ricerca con `findAny()` viene trovato un elemento, i restanti elementi dello stream non vengono esaminati

# esercizio

Quale sarà il risultato se si compila e si esegue il seguente codice?

```
List<String> eroes= Arrays.asList("Sam", "Frodo", "Aragorn");  
boolean flag = eroes.stream().allMatch(str->{  
    System.out.print("Good job: "+str+ " ");  
    return str.equals("Frodo");  
});  
System.out.println(flag);
```

1. Good job: Sam  
false
2. Good job: Sam Good job: Frodo Good job: Aragorn  
false
3. Good job: Sam Good job: Frodo  
true
4. Non compila: l'espressione lambda è scritta male

# esercizio

Quale sarà il risultato se si compila e si esegue il seguente codice?

```
List<String> eroes= Arrays.asList("Sam", "Frodo", "Aragorn");  
boolean flag = eroes.stream().allMatch(str->{  
    System.out.print("Good job: "+str+ " ");  
    return str.equals("Frodo");  
});  
System.out.println(flag);
```

1. **Good job: Sam**  
**false**
2. Good job: Sam Good job: Frodo Good job: Aragorn  
false
3. Good job: Sam Good job: Frodo  
true
4. Non compila: l'espressione lambda è scritta male

Bisogna ricordare che le operazioni di ricerca sono cortocircuitate

# Operazione finale: ricerche

- I metodi di tipo find tornano un oggetto Optional<T>. Ad esempio:

```
Optional<Dish> o = menu.stream()  
                        .filter(Dish::isVegetarian)  
                        .findAny();
```

- ritorna una pietanza (una qualsiasi) vegetariana

```
Dish d = menu.stream().findAny().get(); // ritorna una pietanza (una qualsiasi)  
Dish d = menu.stream().findFirst().get(); //ritorna la prima pietanza
```

- La classe Optional<T> è stata introdotta in Java8 per gestire il tipo di ritorno per quei metodi che prevedono la possibilità di non tornare nessun risultato.
  - Normalmente, il metodo tornerebbe null ma il valore nullo si presta a errori e bug
- Un oggetto Optional rappresenta il risultato o la sua assenza.
  - Potrebbe “contenere” un valore oppure no
  - Se il valore è presente il metodo isPresent() torna true e get() torna il valore



# Optional

E' possibile ottenere un oggetto Optional come risultato di una operazione finale su uno stream ma è possibile anche costruirne uno attraverso i metodi

- static <T> [Optional](#)<T>[empty](#)() → costruisce un oggetto Optional vuoto
- static <T> [Optional](#)<T>[of](#)(T value) → costruisce un oggetto Optional con un valore **non nullo** iniziale (altrimenti `NullPointerException`).
- static <T> [Optional](#)<T>[ofNullable](#)(T value) → costruisce un oggetto Optional con un valore se non nullo, altrimenti costruisce un Optional vuoto

Una volta creato, è possibile invocare i seguenti metodi secondo la logica della pipeline degli stream

- [Optional](#)<T>[filter](#)([Predicate](#)<? super [T](#)> predicate)
- <U> [Optional](#)<U>[flatMap](#)([Function](#)<? super [T](#),[Optional](#)<U>> mapper)
- <U> [Optional](#)<U>[map](#)([Function](#)<? super [T](#),? extends U> mapper)

I metodi value-based come equals, hashCode e toString sono riscritti

# La classe Optional

- I metodi principali sono:
  - **T get()** → ritorna il valore se presente o NoSuchElementException altrimenti
  - **void ifPresent(Consumer<? **super T**> **consumer**)** → se optional contiene un valore esegue la funzione consumer sul valore altrimenti non fa niente
  - **boolean isPresent()** → ritorna true se optional contiene un valore, false in tutti gli altri casi
  - **T orElse(**T** **other**)** → ritorna il valore se presente o other in sua assenza
  - **T orElseGet(Supplier<? **extends T**> **other**)** → ritorna il valore se presente , altrimenti invoca other e ritorna il risultato di questa invocazione
  - **<**X** **extends** Throwable> T orElseThrow(Supplier<? **extends X**> **exceptionSupplier**)** → ritorna il valore se presente , altrimenti ritorna l'exception tornata dall'invocazione su other

# Esempio

```
public static Optional<String> getNazione(String localita){
    Optional<String> option = Optional.empty();
    if("Roma".equals(localita))
        option = Optional.of("Italia");
    else if("Madrid".equals(localita))
        option = Optional.of("Spagna");
    return option;
}

public static void main(String[] args) {
    Optional<String> o1 = getNazione("Roma");
    Optional<String> o2 = getNazione("Atene");

    System.out.println(o1.orElse("Località assente"));
    if(o2.isPresent())
        o2.ifPresent(x -> System.out.println(x));
    else
        System.out.println(o2.orElse("Località assente "));
}
```

**Quale è il risultato?**

# Esempio

```
public static Optional<String> getNazione(String localita){
    Optional<String> option = Optional.empty();
    if("Roma".equals(localita))
        option = Optional.of("Italia");
    else if("Madrid".equals(localita))
        option = Optional.of("Spagna");
    return option;
}

public static void main(String[] args) {
    Optional<String> o1 = getCountry("Roma");
    Optional<String> o2 = getCountry("Atene");

    System.out.println(o1.orElse("Località assente"));
    if(o2.isPresent())
        o2.ifPresent(x -> System.out.println(x));
    else
        System.out.println(o2.orElse("Località assente "));
}
```

**Quale è il risultato?**

Italia  
Localita assente

# La classe Optional

- Esempio di uso di Optional<T>

```
menu.stream()  
    .filter(Dish::isVegetarian)  
    .findAny()  
    .ifPresent(d -> System.out.println(d.getName()));
```

Perché avere 2 metodi molto simili come findFirst() e findAny()?

La differenza sta nella modalità di ricerca: findAny è molto più efficiente perché utilizza il parallelismo in maniera efficiente in quanto non deve tenere conto dell'ordine originale degli elementi letti dalla sorgente.

# Operazione finale: riduzione

Come operazione finale è possibile trarre dallo stream un singolo valore come risultato di una operazione (ad esempio di calcolo)

- In SQL corrisponderebbe alle operazioni di gruppo

Trarre un singolo valore da uno stream significa applicare una certa funzione sugli elementi dello stream in maniera iterativa

- L'operazione viene eseguita come in un for

Ad esempio, se lo stream contiene i numeri {4,5,3,9} si potrebbe pensare ad una funzione di sommatoria che riduce lo stream ad un numero

```
int sum = numbers.stream().reduce(0, (a, b) -> a + b);  
Optional<Integer> sum = numbers.stream().reduce((a, b) -> (a + b));
```

Non essendoci un valore iniziale, se lo stream è vuoto, non c'è risultato

- Il metodo sono i seguenti
  - T reduce(T identity, BinaryOperator<T> accumulator)
  - Optional<T> reduce(BinaryOperator<T> accumulator)
  - <U> U reduce(U identity, BiFunction<U, ? super T, U> accumulator, BinaryOperator<U> combiner)

# Operazione finale: riduzione

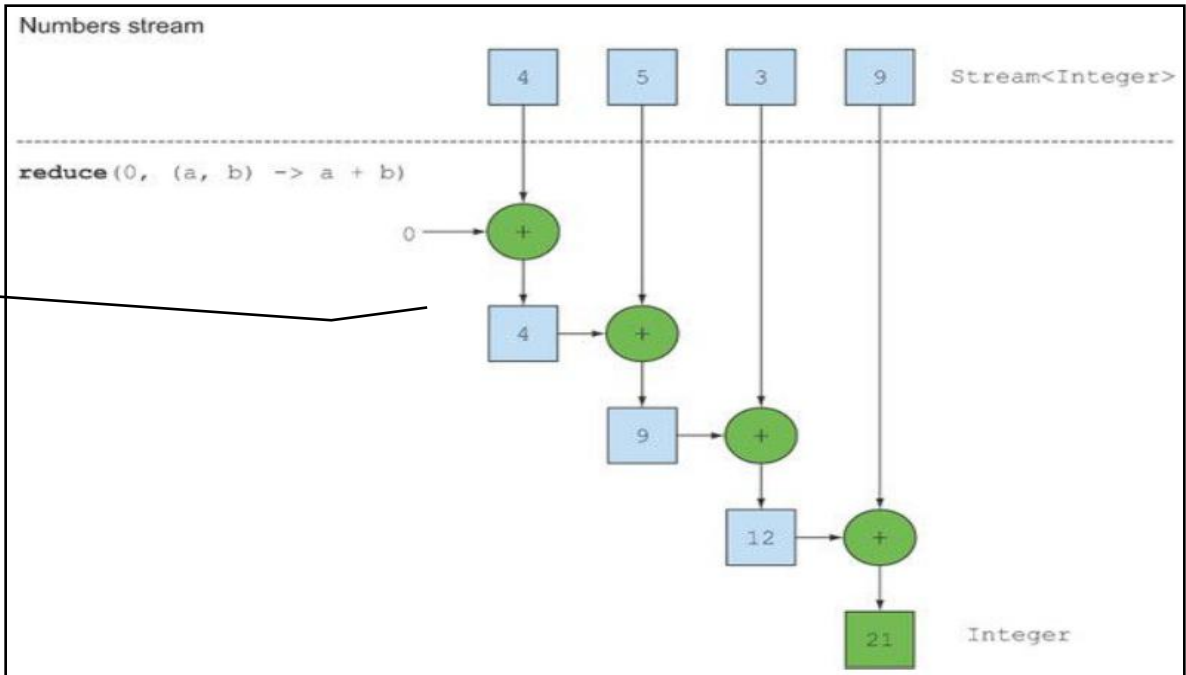
- Esempio di sommatoria:

```
int sum = numbers.stream().reduce(0, (a, b) -> a + b);  
int sum = numbers.stream().reduce(0, Integer::sum); //in alternativa
```

L'operazione Consumer  
sfrutta il concetto di  
accumulazione del dato.

I parametri dell'operazione  
consumer sono il dato  
accumulato e il prossimo  
valore dello stream

Il valore accumulato iniziale  
è il primo parametro del  
metodo (nell'esempio 0)



# Nuovi metodi per Number

- Per agevolare l'uso del metodo reduce sono stati introdotti nuovi metodi nelle classi wrapper numeriche (classi di tipo Number)
- Nelle classi Short, Integer, Long, Float, e Double
  - static int min(int a, int b)
  - static int max(int a, int b)
  - static int sum(int a, int b)



# Operazione finale: riduzione

Esiste anche la versione che usa il parallelismo

```
int sum = numbers.parallelStream().reduce(0, Integer::sum);
```

in questo caso viene utilizzato il framework fork and join per sommare gli elementi secondo una logica “divide et impera”

- Il fork and join usa diversi thread per compiere operazioni parziali.
- Sfrutta il parallelismo reale: di default, apre un numero di thread pari al numero di processori reali della macchina (`Runtime.getRuntime().availableProcessors()`)

ci sono diversi limiti:

- l'operazione deve essere associativa (come la somma), cioè il risultato finale non deve dipendere dall'ordine degli elementi
- Il valore iniziale se presente deve essere “neutro” (zero per la somma)
- l'espressione lambda non deve cambiare di stato, cioè non deve coinvolgere modifiche alle proprietà della classe

Vedere la sezione dedicata alla parallelizzazione

# Operazione finale: count, max, min

- La classe Stream offre un particolare tipo di riduzione:
  - **long count()** → conteggia il numero di elementi nello stream.
  - **Optional<T> max(Comparator<? super T> comparator)** → ritorna il valore massimo in accordo con l'oggetto Comparator passato come parametro
  - **Optional<T> min(Comparator<? super T> comparator)** → ritorna il valore minimo in accordo con l'oggetto Comparator passato come parametro

Nota: i metodi max() e min() sollevano NullPointerException se il massimo ed il minimo sono null (rispettivamente)

Nota: esiste anche average() ma appartiene alle classi Stream specializzate

# esempio

- Dato il seguente codice:

```
List<String> l = Arrays.asList("", "Red", "", "Green", "Black");  
  
long val = l.stream().  
    filter(n -> !n.isEmpty()).  
    count();  
System.out.println(val);
```

stampa il numero di elementi non vuoti dello stream: 3

# Operazione finale: `forEach`

Come operazione finale, `forEach` consente di applicare un certo comportamento a tutti gli elementi dello stream

- `void forEach(Consumer<? super T> action)`

L'ordine non è garantito ne con `stream()` ne con `parallelStream()`

Nel caso si usi un `parallelStream()`, a maggior ragione l'ordine con cui verrà applicata la funzione sugli elementi non è deterministico

- La ripartizione degli elementi nei diversi thread non deve seguire un criterio o un ordine

Per superare questo c'è:

- `void forEachOrdered(Consumer<? super T> action)`

**L'ordine è garantito e le azioni sono fatte una dopo l'altra (senza sovrapporsi)** ma è molto meno performante del precedente

Nel caso si usi `parallelStream()` la ripartizione degli elementi nei diversi thread non deve seguire un criterio o un ordine

# esercizio

Qual è il risultato quando si compila ed esegue questo codice?

```
List<Integer> ls = Arrays.asList(1, 2, 3);  
ls.stream().forEach(System.out::print)  
           .map(a->a*2)  
           .forEach(System.out::print);
```

1. 123
2. 246
3. Errore di compilazione
4. Exception a runtime

# esercizio

Qual è il risultato quando si compila ed esegue questo codice?

```
List<Integer> ls = Arrays.asList(1, 2, 3);  
ls.stream().forEach(System.out::print)  
    .map(a->a*2)  
    .forEach(System.out::print);
```

1. 123
2. 246
3. **Errore di compilazione**
4. Exception a runtime

il metodo `forEach` è una operazione finale che termina la pipeline. Tecnicamente torna `void`

# esempio

- Dato il seguente codice

```
class Test{  
    List<String> lista=null;  
  
    public List<String> getL() {  
        return lista;  
    }  
  
    public void setL(List<String> l) {  
        this.lista = l;  
    }  
    public void stampa(){  
        System.out.println(getL());  
    }  
}
```

**Attenzione alle invocazioni  
static/non static**

```
public static void stampa(String s){  
    System.out.println("ok");  
}
```

```
List<String> l = Arrays.asList("Georg","Micke","Steve");  
Test t = new Test();  
t.setL(l.stream().collect(Collectors.toList()));  
t.getL().forEach(Test::stampa);
```

In base a questa invocazione, il metodo stampa() della classe Test dovrebbe essere statico e prendere una stringa come argomento

# esempio

- Dato il seguente codice

```
class Prodotto{
    private int id;
    private String nome;
    public Prodotto(int id, String nome) {
        this.id = id;
        this.nome = nome;
    }
    public int getId() {return id;}
    public String getNome() {return nome;}
    public String toString() {return nome;}

    static class ProductFilter {
        public static boolean isAvailable(Prodotto p){
            return p.id >= 10;
        }
    }
}
```

## Attenzione alle inner class

Nessun problema qui: il metodo isAvailable() è statico e prende come parametro un Prodotto

Nessun problema qui: il metodo isAvailable() è statico in una classe statica: si invoca attraverso la sintassi Prodotto.ProductFilter.isAvailable()

```
List<Prodotto> l = Arrays.asList(
    new Prodotto(5, "Schedamadre"),
    new Prodotto(20, "Speaker"));

l.stream()
    .filter(Prodotto.ProductFilter::isAvailable)
    .forEach( p -> System.out.println(p));
```



# Operare con gli oggetti immutabili

Bisogna tenere presente che gli oggetti immutabili (soprattutto Wrapper o String) sono immutabili.

Eventuali stream di questo tipo su cui vengono applicate operazioni di modifica dei valori, applicherebbero solo apparentemente queste modifiche

Ad esempio:

```
List<Double> dList = Arrays.asList(10.0, 12.0);  
dList.stream().forEach(x->{ x = x+10; });  
dList.stream().forEach(d->System.out.println(d));
```

vengono creare due stream separati sulla stessa lista di Double, il primo modifica i valori ma solo in apparenza

- Vengono incrementate solo delle copie

il secondo stream stampa i valori [10.0 , 12.0]

# Stateless vs stateful

- Avendo una visione chiara di quello che è possibile fare attraverso gli stream, possiamo classificare le operazioni anche attraverso la loro caratteristica di essere stateless o stateful
  - **Operazione stateless** → per operare NON necessità di mantenere uno stato interno come “memoria” delle operazioni già compiute

Esempio di stateless: map e filter

- **Operazione stateful** → per operare necessità di mantenere uno stato interno come “memoria” delle operazioni già compiute

Esempio di stateful: sum o max

Ad esempio, max deve mantenere (accumulare) lo stato corrente del valore massimo per confrontarlo con i vari elementi dello stream

# Mantenimento dello stato

- Nell'ambito del mantenimento dello stato, bisogna tenere presente che non è possibile modificare una variabile locale in una espressione lambda
- Ad esempio, il seguente codice:

```
List<Integer> ls = Arrays.asList(1, 2, 3);  
double sum = 0;  
ls.stream().forEach(a->{ sum=sum+a; });
```

**provoca un errore di compilazione:** la variabile `sum` non può essere modificata nella espressione lambda in quanto va considerata `final`

- in inglese: “only final or effective final local variables can be used in a lambda expression”
- Il seguente codice invece è perfettamente legale:

```
List<String> listaGeneri= new ArrayList<>();  
books.stream().map(Book::getGenere).forEach(s-> listaGeneri.add(s));  
System.out.println(listaGeneri);
```

aggiungere un elemento ad una lista dichiarata come variabile locale non significa violarne le sue proprietà di costante `final`

# Bounded vs unbounded

- Altro modo di classificare gli stream è la loro capacità di prevedere la quantità di spazio necessario per il risultato
  - **Bounded** → una quantità di spazio fisso, indipendentemente da quanti elementi vengono processati nello stream

Esempio: sum o max

- **Unbounded** → una quantità di spazio non determinabile a priori e che dipende da quanti elementi vengono processati nello stream

Esempio: filter o distinct

# Tabella riassuntiva

| Operation | Type         | state e bound        | Return type | Type/functional Interface user | Function descriptor              |
|-----------|--------------|----------------------|-------------|--------------------------------|----------------------------------|
| filter    | intermediate |                      | Stream<T>   | Predicate<T>                   | $T \rightarrow \text{boolean}$   |
| distinct  | intermediate | (stateful-unbounded) | Stream<T>   |                                |                                  |
| skip      | intermediate | (stateful-bounded)   | Stream<T>   | long                           |                                  |
| limit     | intermediate | (stateful-bounded)   | Stream<T>   | long                           |                                  |
| map       | intermediate |                      | Stream<R>   | Function<T, R>                 | $T \rightarrow R$                |
| flatMap   | intermediate |                      | Stream<R>   | Function<T, Stream<R>>         | $T \rightarrow \text{Stream}<R>$ |
| sorted    | intermediate | (stateful-unbounded) | Stream<T>   | Comparator<T>                  | $(T, T) \rightarrow \text{int}$  |
| anyMatch  | terminal     |                      | boolean     | Predicate<T>                   | $T \rightarrow \text{boolean}$   |
| allMatch  | terminal     |                      | boolean     | Predicate<T>                   | $T \rightarrow \text{boolean}$   |
| noneMatch | terminal     |                      | boolean     | Predicate<T>                   | $T \rightarrow \text{boolean}$   |
| findAny   | terminal     |                      | Optional<T> |                                |                                  |
| findFirst | terminal     |                      | Optional<T> |                                |                                  |
| forEach   | terminal     |                      | void        | Consumer                       | $T \rightarrow \text{void}$      |
| collect   | terminal     |                      | R           | Collector<T,A,R>               |                                  |
| reduce    | terminal     |                      | Optional<T> | BinaryOperator<T>              | $(T, T) \rightarrow T$           |
| count     | terminal     |                      | long        |                                |                                  |
| max       | terminal     |                      | Optional<T> | Comparator<T>                  | $(T, T) \rightarrow \text{int}$  |
| min       | terminal     |                      | Optional<T> | Comparator<T>                  | $(T, T) \rightarrow \text{int}$  |

# Specializzazioni

- Le interfacce funzionali offerte da Java 8 utilizzano i generics per aumentare le possibilità di utilizzo.
- L'uso dei generics però esclude i primitivi che possono essere utilizzati **solo** attraverso i wrapper Java (grazie all' autoboxing)
  - L'autoboxing usa implicitamente i wrapper
- Usare i wrapper esplicitamente o usare la tecnica del boxing ha un costo computazionale che potrebbe essere non trascurabile.
- Per evitare l'uso dei wrapper, Java8 offre **Stream specializzati** che usano **interfacce funzionali specializzate** ma solo per i tipi **int, long e double**

|                           |                                                         |                                                                                                                |
|---------------------------|---------------------------------------------------------|----------------------------------------------------------------------------------------------------------------|
| <b>Predicate&lt;T&gt;</b> | <b>IntPredicate, LongPredicate,<br/>DoublePredicate</b> | Ad esempio, per la generica interfaccia Predicate<T> vi sono le interfacce specializzate si int, double e long |
|---------------------------|---------------------------------------------------------|----------------------------------------------------------------------------------------------------------------|

# Specializzazioni

Per facilitare l'uso degli stream con i tipi primitivi, Java8 mette a disposizione specializzazioni delle classi Stream per i tipi primitivi

- **IntStream**
- **DoubleStream**
- **LongStream**

Questi stream manipolano più efficientemente i dati e offrono metodi specifici per i rispettivi valori primitivi. Ad esempio:

```
int calories = menu.stream()  
                    .map(Dish::getCalories)  
                    .sum();
```

**per quanto sensato non compila** perché il metodo map() ritorna uno Stream<T> che non possiede il metodo sum(). Il seguente:

```
int calories = menu.stream()  
                    .mapToInt(Dish::getCalories)  
                    .sum();
```

invece compila e funziona come ci aspettiamo perché mapToInt torna un IntStream che possiede il metodo sum()

# Specializzazioni

- Le classi di specializzazione hanno metodi specializzati
  - static [IntStream](#) [of](#)(int... values)
  - static [IntStream](#) [of](#)(int t)
  - ....(stesso per Long e Double)
- Gli stream specializzati forniscono altre operazioni finali:
  - [OptionalDouble](#) [average\(\)](#) → calcola la media dei valori dello stream



# Specializzazioni

- Le classi `IntStream` e `LongStream` forniscono anche un metodo per generare un range di valori
  - `rangeClosed(int from, int to)`** → ritorna uno `IntStream` con tutti i valori interi tra `from` e `to` (`from` e `to` inclusi)
  - `range(int from, int to)`** → ritorna uno `IntStream` con tutti i valori interi tra `from` e `to` (`from` incluso e `to` escluso)

Esempio di `rangeClosed()`

```
IntStream stream = IntStream.rangeClosed(1,100)
                                .filter(n -> n % 2==0);

int calorie = maxCalorie.orElse(stream.count());
```

si ottiene uno stream con i numeri pari. Il numero di elementi nello stream è pari a 50

# Specializzazioni

In generale, vi sono metodi che consentono di convertire uno stream base in uno stream di primitivi e viceversa

- [IntStream](#) [mapToInt](#)([ToIntFunction](#)<? super [I](#)> mapper) → converte uno stream generico in un IntStream
- [DoubleStream](#) [mapToDouble](#)([ToDoubleFunction](#)<? super [I](#)> mapper) → converte uno stream generico in un DoubleStream
- [LongStream](#) [mapToLong](#)([ToLongFunction](#)<? super [I](#)> mapper) → converte uno stream generico in un LongStream

Analoghe operazioni con flatMap

- [DoubleStream](#) [flatMapToDouble](#)([Function](#)<? super [I](#),? extends [DoubleStream](#)> mapper)
- [IntStream](#) [flatMapToInt](#)([Function](#)<? super [I](#),? extends [IntStream](#)> mapper)
- [LongStream](#) [flatMapToLong](#)([Function](#)<? super [I](#),? extends [LongStream](#)> mapper)
- **boxed()** → converte uno stream specializzato in un generico Stream

esempio di uso di boxed()

```
Stream<Integer> = menu.stream()  
                    .mapToInt(Dish::getCalories)  
                    .boxed();
```

# Specializzazioni

Esiste anche una versione specializzata della classe `Optional` da utilizzare insieme agli stream specializzati.

- **OptionalInt** → Il metodo `get` qui si chiama `getAsInt()` ma il comportamento è lo stesso
- **OptionalDouble** → Il metodo `get` qui si chiama `getAsDouble()` ma il comportamento è lo stesso
- **OptionalLong** → Il metodo `get` qui si chiama `getAsLong()` ma il comportamento è lo stesso

Esempio di utilizzo di `OptionalInt`

```
OptionalInt maxCalorie= menu.stream()  
                           .mapToInt(Dish::getCalories)  
                           .max();  
  
int calorie = maxCalorie.orElse(1);
```

il codice calcola il massimo valore per lo stream di interi e se non c'è fornisce come massimo di default il valore 1

NOTA: anche i metodi delle classi specializzate sono specializzati

- Ad esempio, invece di `get()` c'è `getAsDouble`

# Costruzione degli Stream

- Fino ad ora abbiamo costruito uno stream a partire da una collezione di dati già esistente. E' possibile però costruire stream, compresi quelli specializzati, in diversi modi ed a partire da diverse origini
  - **Stream a partire dai dati** → uno stream costruito a partire da un elenco di dati
    - Metodi statici della classe Stream e specializzazioni in IntStream, DoubleStream e LongStream
  - **Stream a partire da un array** → uno stream costruito a partire da un elenco di dati presenti in un array
    - Metodi statici della classe Arrays e specializzazioni in overloading per IntStream, DoubleStream e LongStream
  - **Stream a partire da un collection** → uno stream costruito a partire da un elenco di dati presenti in un collection
    - Metodi di default presenti in Collection e quindi ereditati da tutti i tipi di collection. Mancano le specializzazioni
  - **Stream a partire da un file** → uno stream costruito a partire da un file
    - Metodi statici della classe Files
  - **Stream a partire da una funzione** → uno stream di infiniti dati costruito in base ad una funzione
    - Metodi statici della classe Stream e specializzazioni in IntStream, DoubleStream e LongStream

# Stream a partire dai dati

La classe Stream mette a disposizione il metodo statico of() in cui è possibile fornire un elenco (da 0 a n) di valori.

- **static <T> Stream<T> of(T... values)**
- **static <T> Stream<T> of(T t)**

Ad esempio:

```
Stream<String> stream = Stream.of("Java 8 ", "Lambdas ", "In ", "Action");  
  
stream.map(String::toUpperCase)  
      .forEach(System.out::println);
```

stampa l'elenco dei valori passai al metodo of().

E' possibile anche creare uno stream vuoto con l'apposito metodo empty()

- **static <T> Stream<T> empty()**

```
Stream<String> emptyStream = Stream.empty();
```

E' possibile anche ottenere uno stream come concatenazione di altri 2

- **static IntStream concat(IntStream a, IntStream b)**

# Stream a partire da un array

- La classe Arrays mette a disposizione il metodo statico stream() in cui è possibile fornire un elenco sotto di valori sotto forma di vettore.

Ad esempio è possibile convertire un vettore di interi in un IntStream:

```
int[] numbers = {2, 3, 5, 7, 11, 13};  
int sum = Arrays.stream(numbers).sum();  
System.out.println(sum);
```

stampa la sommatoria dei valori calcolati dal metodo sum()

# Stream a partire da una collection

L'interfaccia Collection mette a disposizione i metodi di default stream() e parallelStream() per “agganciare” i dati presenti in una collection ad uno nuovo stream

- `default Stream<E> stream()` //singolo thread
- `default Stream<E> parallelStream()` //multiThread

Ad esempio è possibile convertire un Set di Integer in uno Stream:

```
Set<Integer> set = new HashSet<>();  
Stream<Integer> st = set.stream();
```

notare che non esistono i metodi per fornire una versione specializzata degli stream.  
Se si vuole una versione specializzata bisogna utilizzare le apposite operazioni intermedie della pipeline

```
Set<Integer> set = new HashSet<>();  
IntStream st = set.stream().mapToInt(i -> i);
```

La lambda expression dovrebbe essere  
**i -> i.intValue()**  
ma grazie al boxing/unboxing possiamo scrivere anche solo  
**i -> i**

# Stream a partire da un file

- La classe Files introdotta con Java NIO mette a disposizione diversi metodi statici che costruiscono uno stream a partire da un file
- Questi stream, leggendo dal file system possono sollevare errori di tipo IO
- Ad esempio il metodo lines() restituisce uno stream di tipo string i cui elementi sono le singole righe di un file di testo

```
try(Stream<String> lines = Files.lines(Parh.get("data.txt"), Charset.defaultCharset())){  
    //uso dello stream  
}
```



# Stream a partire da una funzione

Creare uno stream a partire da una funzione significa definire una funzione che determina una serie di valori. Lo stream creato a partire da questa funzione fornisce la serie di valori calcolati dalla funzione.

In questo senso non c'è un limite massimo di valori o un set di valori predeterminato; questo motivo si chiamano **infinite stream** e sono unbounded

La classe Stream possiede i metodi statici: **iterate()** e **generate()**. Ad esempio

```
Stream.iterate(0, n -> n + 2)
    .limit(10)
    .forEach(System.out::println);
```

stampa i primi 10 valori pari a partire da 0

Nota: il metodo `limit()` è utile per impostare il limite delle operazioni sullo stream per evitare che il `forEach` continui a ciclare all'infinito

# Stream a partire da una funzione

## Iterate (unbounded stateful)

Il metodo `iterate()` genera infiniti valori; ogni nuovo valore viene creato applicando la funzione al valore calcolato in precedenza

La logica dei parametri del metodo `iterate` è lo stesso del metodo `reduce` visto in precedenza; i parametri sono 2: il primo è il valore iniziale ed il secondo è `UnaryOperator<T>`

Esempio complesso con uso dei vettori. La sequenza di fibonacci

```
Stream.iterate(new int[]{0, 1}, t -> new int[]{t[1], t[0]+t[1]})  
    .limit(20)  
    .forEach(t -> System.out.println("(" + t[0] + "," + t[1] + ")"));
```

per quanto possa sembrare strano, `iterate` si comporta come l'esempio precedente dei numeri pari.

# Stream a partire da una funzione

## generate (unbounded stateless o stateful)

Il metodo generate() non invoca la funzione sul valore calcolato in precedenza; in generale si comporta come iterate e produce un infinite stream unbounded; la differenza è il secondo parametro che è di tipo Supplier<T>

In pratica, ogni nuovo valore viene generato invocando nuovamente la funzione. In questo esempio:

```
Stream.generate(Math::random)
    .limit(5)
    .forEach(System.out::println);
```

Per quanto l'oggetto Random costruito sia uno solo, si ottengono e si stampano 5 numeri random diversi. ogni numero generato non dipende dal precedente così da avere una logica stateless

# Stream a partire da una funzione

Applicando una logica più complessa è possibile utilizzare il metodo `generate()` con una logica stateful

L'esempio precedente è valido perché il metodo `random` non riceve parametri e torna un valore, cioè risponde ai requisiti della chiamata lambda di `Supplier<T>: () → T`

Per gli stessi motivi, anche il seguente è valido:

```
IntStream twos = IntStream.generate(new IntSupplier(){
    int count = 0;
    public int getAsInt(){
        count++;
        return count;
    }
});
```

è possibile mantenere lo stato nelle proprietà della classe rendendo la funzione stateful.

Questo perché bisogna ricordare che:

- Verrà costruita una sola istanza della sottoclasse anonima. Quindi le variabili di istanza fanno da accumulatore
- Verrà invocato `n` volte il metodo di generazione

# esercizio

Dato il seguente codice:

```
List<Integer> l1 = Arrays.asList(1,2,3,4);  
List<Integer> l2 = Arrays.asList(5,6,7,8);  
// inserire qui
```

quale delle seguenti soluzioni consente di stampare 1,2,3,4,5,6,7,8?

1. `Stream.of(l1,l2)`  
    `.flatMapToInt(list -> list.stream())`  
    `.forEach(s -> System.out.print(s + " "));`
2. `Stream.of(l1,l2)`  
    `.flatMap(list -> list.intStream())`  
    `.forEach(s -> System.out.print(s + " "));`
3. `Stream.of(l1,l2)`  
    `.flatMap(list -> list.stream())`  
    `.forEach(s -> System.out.print(s + " "));`
4. `Stream.of(l1,l2)`  
    `.flatMap(list -> list.stream()).flatMap(list -> list.stream())`  
    `.forEach(s -> System.out.print(s + " "));`

# esercizio

Dato il seguente codice:

```
List<Integer> l1 = Arrays.asList(1,2,3,4);  
List<Integer> l2 = Arrays.asList(5,6,7,8);  
// inserire qui
```

quale delle seguenti soluzioni consente di stampare 1,2,3,4,5,6,7,8?

1. `Stream.of(l1,l2)`  
    `.flatMapToInt(list -> list.stream())`  
    `.forEach(s -> System.out.print(s + " "));`
2. `Stream.of(l1,l2)`  
    `.flatMap(list -> list.intStream())`  
    `.forEach(s -> System.out.print(s + " "));`
3. **`Stream.of(l1,l2)`**  
    **`.flatMap(list -> list.stream())`**  
    **`.forEach(s -> System.out.print(s + " "));`**
4. `Stream.of(l1,l2)`  
    `.flatMap(list -> list.stream()).flatMap(list -> list.stream())`  
    `.forEach(s -> System.out.print(s + " "));`

# Uso avanzato degli stream

Parallelismo e riduzioni avanzate

# Parallel, sequential, unordered

E' possibile definire uno stream le cui operazioni siano eseguire in una logica monothread o multithread

- Perfettamente in linea con le politiche dichiarative, dobbiamo solamente indicare la nostra scelta: utilizzare uno stream parallelo invece che sequenziale o viceversa.
- [S parallel\(\)](#) → marca lo stream come parallelo e ritorna uno stream equivalente. Ritorna se stesso se già parallelo
- [S sequential\(\)](#) → marca lo stream come sequenziale e ritorna uno stream equivalente. Ritorna se stesso se già sequenziale

Eventualmente c'è anche

- [S unordered\(\)](#) → marca lo stream come unordered e ritorna uno stream equivalente. Ritorna se stesso se già unordered o se marcato come unordered

Uno stream unordered non deve tenere conto della sequenza degli elementi provenienti dalla sorgente

- Alcune sorgenti sono già così, come ad esempio un HashSet



# Ordered/unordered

La combinazione ordered/unordered e parallel/sequential è complessa ma le domande non dovrebbero creare combinazioni troppo complesse

Su **ordered/unordered** bisogna tenere conto che:

- Il fatto che uno stream tratti gli elementi in un certo ordine o meno dipende dalla sorgente e dalle operazioni intermedie.
  - Alcune sorgenti (come List o array) sono ordinate intrinsecamente, mentre altre (come HashSet) no
  - Alcune operazioni intermedie, come sort(), possono imporre un ordine su un flusso altrimenti non ordinato, e altre possono rendere un flusso da ordinato a non ordinato, come unordered().
- Alcune operazioni final possono ignorare l'ordine degli elementi, come forEach ().
- Se un flusso è ordinato, la maggior parte delle operazioni è vincolata a operare sugli elementi nel loro ordine;
  - Ad esempio, se la sorgente di un flusso è una lista contenente [1, 2, 3], il risultato dell'esecuzione di map ( $x \rightarrow x * 2$ ) deve essere [2, 4, 6]. Tuttavia, se la sorgente non ha un ordine di incontro definito, qualsiasi permutazione dei valori [2, 4, 6] sarebbe un risultato valido.

# Parallel, sequential

Una volta creato uno stream si può marcare come parallelo o sequenziale. Ad esempio:

```
stream.parallel().reduce();
```

E' possibile marcare uno stream come parallelo o sequenziale anche in momenti diversi della pipeline

```
stream  
    .parallel()  
    .filter(...)  
    .sequential()  
    .map(...)  
    .parallel()  
    .reduce();
```

Nota: di per se, l'invocazione a questi metodi modifica solo una variabile booleana interna

# Parallel, sequential

- Esempio. Supponiamo di voler realizzare la sommatoria dei primi n numeri naturali. Il codice prima dell'introduzione degli stream sarebbe:

```
public static long iterativeSum(long n) {  
    long result = 0;  
    for (long i = 1L; i <= n; i++) {  
        result += i;  
    }  
    return result;  
}
```

- Per ottenere lo stesso risultato con uno stream sequenziale il codice potrebbe essere il seguente:

```
public static long iterativeSum(long n) {  
    long result = Stream.iterate(1L, i = i + 1)  
                        .limit(n)  
                        .reduce(0L, Long::sum);  
    return result;  
}
```

# Parallel, sequential

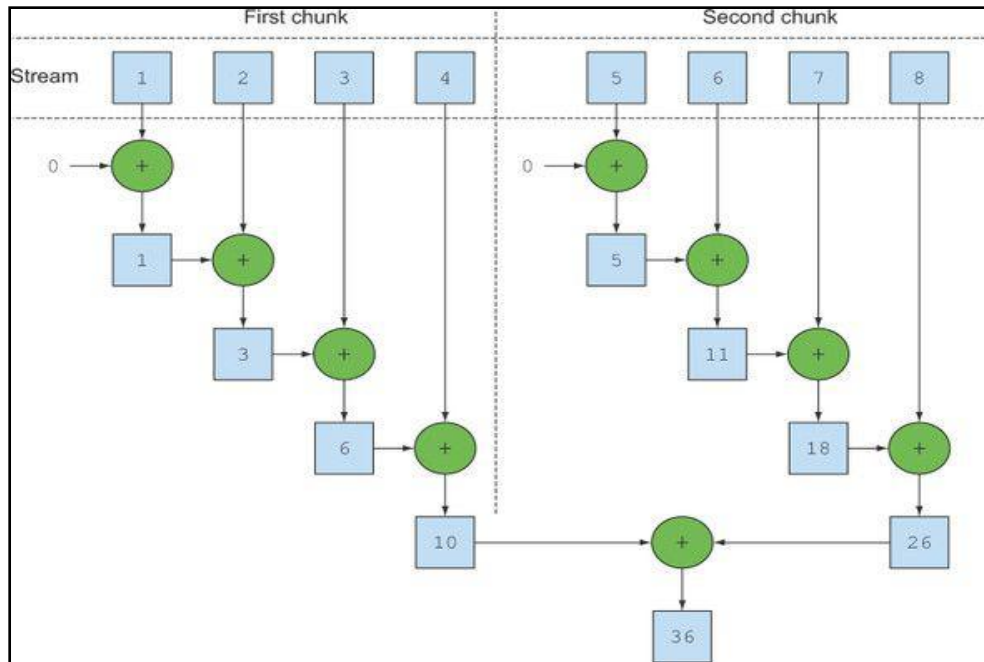
- Si potrebbe pensare di compiere l'operazione di riduzione attraverso uno stream parallelo

```
public static long iterativeSum(long n) {  
    long result = Stream  
        .iterate(1L, i = i + 1)  
        .limit(n)  
        .parallel()  
        .reduce(0L, Long::sum);  
    return result;  
}
```

Quello che accade internamente è una suddivisione del lavoro in chunks indipendenti e paralleli  
I diversi chunks si ricombinano nel finale per restituire il risultato.

**Nel caso di reduce**, va notato che il valore **identity**, cioè il primo parametro della riduzione viene utilizzato in ogni chunk, **quindi deve essere neutro**

**Reduce mantiene la sequenza originale della sorgente**



# Esempio

Dato il seguente codice:

```
List<String> vals = Arrays.asList("a", "b");  
String join = vals  
    .parallelStream()  
    .reduce("_", (a, b)->a.concat(b));  
System.out.println(join);
```

quale è il possibile output?

\_ab

\_a\_b

\_a\_b oppure \_ab

\_b\_a

\_ab oppure \_ba

Una qualsiasi tra: \_ab, \_ba, \_a\_b, \_b\_a

# Esempio

Dato il seguente codice:

```
List<String> vals = Arrays.asList("a", "b");
String join = vals
    .parallelStream()
    .reduce("_", (a, b)->a.concat(b));
System.out.println(join);
```

quale è il possibile output?

1. `_ab` // in teoria anche questa con una macchina single-core
2. `_a_b`
3. `_a_b` oppure `_ab`
4. `_b_a`
5. `_ab` oppure `_ba`
6. Una qualsiasi tra: `_ab`, `_ba`, `_a_b`, `_b_a`

Considerato che la sorgente è `ordered` e non è stato invocato il metodo `unordered()`, possiamo concludere che la `a` non potrà mai essere concatenata prima della `b`

Inoltre, sebbene in caso di macchina single-core lo stream potrebbe restare sequenziale (nonostante l'invocazione al metodo `parallelStream()`) **l'esame sembra non contemplare questa possibilità**

# Esempio

- Dato il seguente codice:

```
List<String>l = Arrays.asList("a ","b ","c");  
BinaryOperator<String> b = (s1, s2) -> s1.concat(s2);  
String sen = l.stream().reduce("concat ", b);  
System.out.println(sen);
```

quale è il possibile output?

1. concat a concat b concat c
2. concat a b c
3. Qualsiasi combinazione di concat a concat b concat c
4. concat seguito da qualsiasi combinazione di a b c

# Esempio

Dato il seguente codice:

```
List<String>l = Arrays.asList("a ", "b ", "c");  
BinaryOperator<String> b = (s1, s2) -> s1.concat(s2);  
String sen = l.stream().reduce("concat ", b);  
System.out.println(sen);
```

quale è il possibile output?

1. concat a concat b concat c
- 2. concat a b c**
3. Qualsiasi combinazione di concat a concat b concat c
4. concat seguito da qualsiasi combinazione di a b c

in questo caso non ci sono dubbi sul risultato: la lista è ordered e lo stream è sequenziale



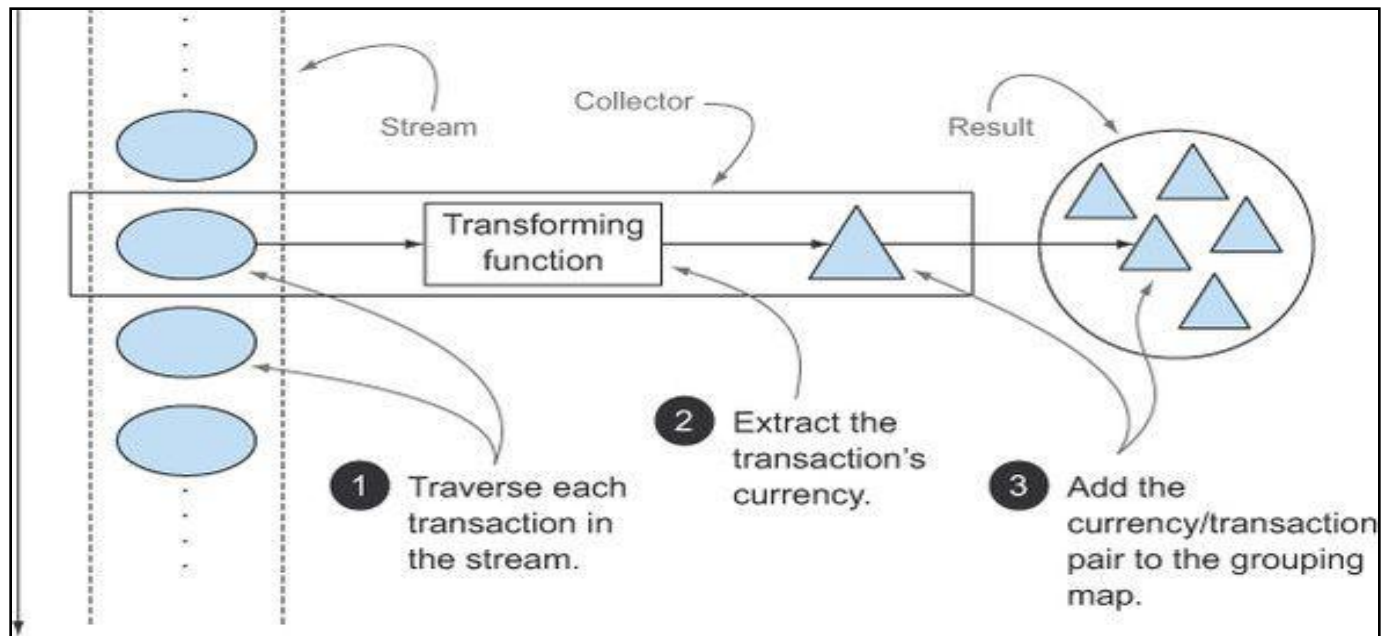
# Operazione finale: collect

- Di base, il metodo `collect()` trasforma i dati presenti in uno stream in una collection (cioè trasforma uno Stream in una Collection o Map)
- L'operazione però va vista come una forma di riduzione (metodo `reduct()`) perché gli elementi dello stream **vengono processati** prima di essere collocati nella collection finale
  - L'operazione di processing è definita formalmente dall'interfaccia **Collector**
  - In una semplice trasformazione, come ad esempio `collect(toList())`, l'operazione di processing non c'è (si raccolgono solo i dati) e quindi non vi sono effetti particolari
- E' possibile definire riduzioni complesse che equivalgono alle operazioni di raggruppamento di SQL (clausola Group By di una Select oppure funzioni di gruppo SQL)

# Operazione finale: collect

```
//import static di Collectors
```

```
Map<Currency, List<Transaction>> transactionsByCurrencies =  
transactions.stream()  
    .collect(groupingBy(Transaction::getCurrency));
```



# Operazione finale: collect

- La classe **Collectors** possiede diversi metodi statici allo scopo di applicare una logica di processing predefinita. I principali sono:
  - `toList()` → nessuna operazione di processing.
  - `counting()` → torna il numero di elementi presenti nello stream
  - `groupingBy()` → raggruppamento in base ad un valore
  - `maxBy()` → tornano l'elemento massimo dello stream in base ad un comparatore
  - `minBy()` → tornano l'elemento minimo dello stream in base ad un comparatore
  - `summingInt()`, `summingDouble()`, `summingLong()`
  - `summarizingInt()`, `summarizingDouble()`, `summarizingLong()`
  - `averagingInt()`, `averagingDouble()`, `averagingLong()`
  - `joining()` → concatena tutti gli elementi di uno stream in una unica stringa
  - `toMap()` → accumula elementi in una mappa le cui chiavi e valori sono definite da altrettante funzioni passate come parametro
  - `averagingDouble()`, `averagingLong()`, `averagingInt()` → calcola la media dei valori

Nota: sono tutte varianti convenienti del più generale metodo `reduce()`, **quindi vanno viste come particolari operazioni di riduzione**

# Esempi di collect con logica

- `IntSummaryStatistics menuStatistics = menu.stream().collect(summarizingInt(Dish::getCalories));`
- `int totalCalories = menu.stream().collect(summingInt(Dish::getCalories));`
- `double avgCalories = menu.stream().collect(averagingInt(Dish::getCalories));`
- `long howManyDishes = menu.stream().collect(Collectors.counting());`
- `Comparator<Dish> dishCaloriesComparator = Comparator.comparingInt(Dish::getCalories);`  
`Optional<Dish> mostCalorieDish = menu.stream().collect(maxBy(dishCaloriesComparator));`
- `String shortMenu = menu.stream().map(Dish::getName).collect(joining());`
- `String shortMenu = menu.stream().map(Dish::getName).collect(joining(", "));`

# toMap

L'operazione toMap è particolarmente complessa ed offre la possibilità di trasformare uno stream in una Map.

- Le chiavi ed i corrispondenti valori sono determinati da funzioni passate come parametro.

I metodi sono:

- public static <T,K,U> Collector<T,?,Map<K,U>> toMap**(  
Function<? super T,? extends K> keyMapper,  
Function<? super T,? extends U> valueMapper)
- public static <T,K,U> Collector<T,?,Map<K,U>> toMap**(  
Function<? super T,? extends K> keyMapper,  
Function<? super T,? extends U> valueMapper,  
BinaryOperator<U> mergeFunction)
- public static <T,K,U,M extends Map<K,U>> Collector<T,?,M> toMap**(  
Function<? super T,? extends K> keyMapper,  
Function<? super T,? extends U> valueMapper,  
BinaryOperator<U> mergeFunction,  
Supplier<M> mapSupplier)

Le due funzioni per la determinazione delle chiavi e dei valori. Se si producono chiavi duplicate viene sollevata una exception

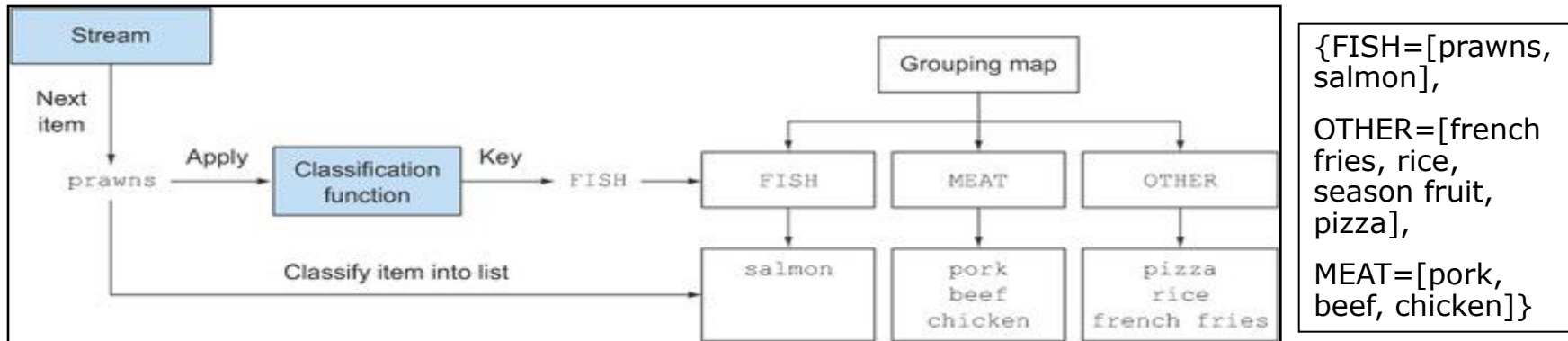
Il terzo parametro BinaryOperator è utile per risolvere le collisioni tra le chiavi come funzione dei valori

Il quarto parametro Supplier consente di creare una nuova mappa in cui riversare il risultato

# grouping

- Raggruppare in base ad un criterio è una operazione molto comune sui dati
  - Ad esempio, in una select SQL corrisponderebbe alla clausola Group By
- A tale scopo, la classe Collectors mette a disposizione il metodo groupingBy() da utilizzare insieme a collect().
  - Il metodo prende una Function e ritorna una mappa le cui chiavi sono il criterio di raggruppamento mentre i valori solo gli elementi appartenenti a quel raggruppamento.
  - Ad esempio:

```
Map<Dish.Type, List<Dish>> dishesByType = menu.stream().collect(groupingBy(Dish::getType));
```



```
{FISH=[prawns, salmon],  
OTHER=[french fries, rice, season fruit, pizza],  
MEAT=[pork, beef, chicken]}
```

# grouping

- Considerato che il parametro di questo metodo è `Supplier<T>` è possibile passare una espressione lambda complessa. Ad esempio:

```
public enum CaloricLevel { DIET, NORMAL, FAT }

Map<CaloricLevel, List<Dish>> dishesByCaloricLevel = menu.stream().collect(
    groupingBy(dish -> {
        if (dish.getCalories() <= 400) return CaloricLevel.DIET;
        else if (dish.getCalories() <= 700) return CaloricLevel.NORMAL;
        else return CaloricLevel.FAT;
    }
));
```

ritorna le pietanze raggruppate in base agli elementi dell'enum `CaloricLevel`. Il criterio di attribuzione di una pietanza ad un elemento dell'enum è calcolato dalla espressione lambda

# esempio

- Dato il seguente codice

```
public class Studente {  
    private String nome, citta, corso;  
  
    public Studente(String nome, String citta, String corso) {  
        this.nome = nome; this.citta = citta; this.corso = corso;  
    }  
    public String getNome() { return nome;}  
    public String getCitta() { return citta;}  
    public String getCorso() { return corso;}  
  
    public String toString() {  
        return nome + " : " + citta + " : " + corso;  
    }  
}
```

```
List<Studente> l = Arrays.asList(  
    new Studente("Mauro", "Napoli", "JavaSE11"),  
    new Studente("Sergio", "Roma", "JavaSE8"),  
    new Studente("Pietro", "Milano", "JavaSE11"));  
  
l.stream().collect(Collectors.groupingBy(Studente::getCorso))  
    .forEach((key, value) -> System.out.println(value));
```

Il metodo collect ritorna una Map. La classe concreta utilizzata è HashMap.

In generale, il metodo forEach, invocato su una mappa consuma secondo l'iteratore della mappa

In questo caso, essendo un HashMap, l'iteratore è tale che l'ordine non è garantito

Notare che i valori potrebbero non avere un ordinamento sulla chiave

[Mauro : Napoli : JavaSE8,  
Pietro : Milano : JavaSE8]

[Sergio : Roma : JavaSE7]



# Grouping multilivello

- Proprio come in SQL, è possibile definire un raggruppamento su 2 o più livelli. Ad esempio, vogliamo raggruppare le pietanze in base al tipo e al livello di calore (in questo ordine)

```
Map<Dish.Type, Map<CaloricLevel, List<Dish>>> dishesByTypeCaloricLevel =  
    menu.stream().collect(  
        groupingBy(Dish::getType,  
            groupingBy(dish -> {  
                if (dish.getCalories() <= 400) return CaloricLevel.DIET;  
                else if (dish.getCalories() <= 700) return CaloricLevel.NORMAL;  
                else return CaloricLevel.FAT;  
            })  
        )  
    );
```

First-level classification function

Second-level classification function

notare il tipo di ritorno: **Map<Dish.Type, Map<CaloricLevel, List<Dish>>>**

```
{  
    MEAT={DIET=[chicken], NORMAL=[beef], FAT=[pork]},  
    FISH={DIET=[prawns], NORMAL=[salmon]},  
    OTHER={DIET=[rice, seasonal fruit], NORMAL=[french fries, pizza]}  
}
```

# Logica multilivello

- Uno dei metodi `groupBy` prende come parametro in generale un altro metodo con logica e non necessariamente `broupingBy()`
- Ad esempio:

```
Map<Dish.Type, Long> typesCount = menu.stream().collect(  
    groupBy(Dish::getType, counting());
```

si ottengono le pietanze suddivise in base al tipo ed il numero di pietanze per ogni tipo

```
{MEAT=3, FISH=2, OTHER=4}
```

nota il metodo

- `groupBy(f)`
- `groupBy(f, toList())`

Sono equivalenti